

AI SOCIAL PIPELINE

Architecture Overview

How our self-serve generator is built, how a request runs start to finish, where files go, and how concurrent requests are handled.

Stack & State

Sequencing

File Handoff

Concurrency

What it's built with

The whole pipeline lives in a managed workflow engine — no local codebase, no server to babysit. It runs on n8n Cloud and talks to two external APIs.



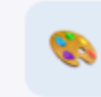
n8n Cloud

The engine. One workflow (76 nodes) handles routing, generation, and delivery. Managed hosting — nothing runs on a desktop.



Slack

A slash command opens a Block Kit modal — structured input (output type, situation, formats), not parsed free text.



kie.ai

One API for everything: nano-banana still images, Kling image-to-video, and a Gemini vision model for the quality check.



How one request is remembered

Each submission is its own n8n execution with persisted state — the input travels through the workflow as data, not as a shared in-memory object. A restart never loses an in-flight request, and two requests never touch the same state.

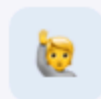
01.5 · DESIGN RATIONALE

Why it's built this way



Managed, not local

It runs unattended on hosted infrastructure. No machine to leave switched on, and execution state survives restarts.



Self-serve for support

Built so the support team generates on-brand assets themselves, with no involvement from the art team per request.



Guardrails first

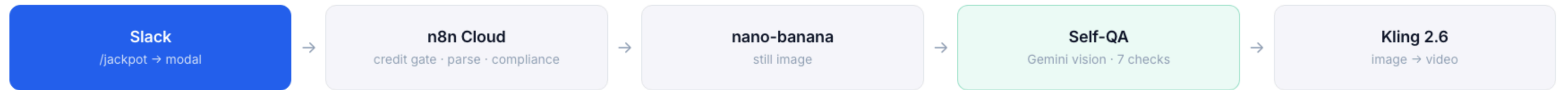
Because it is unattended and this is a social-casino product, cost, access, and compliance checks are wired in before any spend.

The trade-off: a managed engine is less flexible than custom code, but it removes the two things that break unattended tools — a single host, and shared mutable state.



02 · SYSTEM MAP

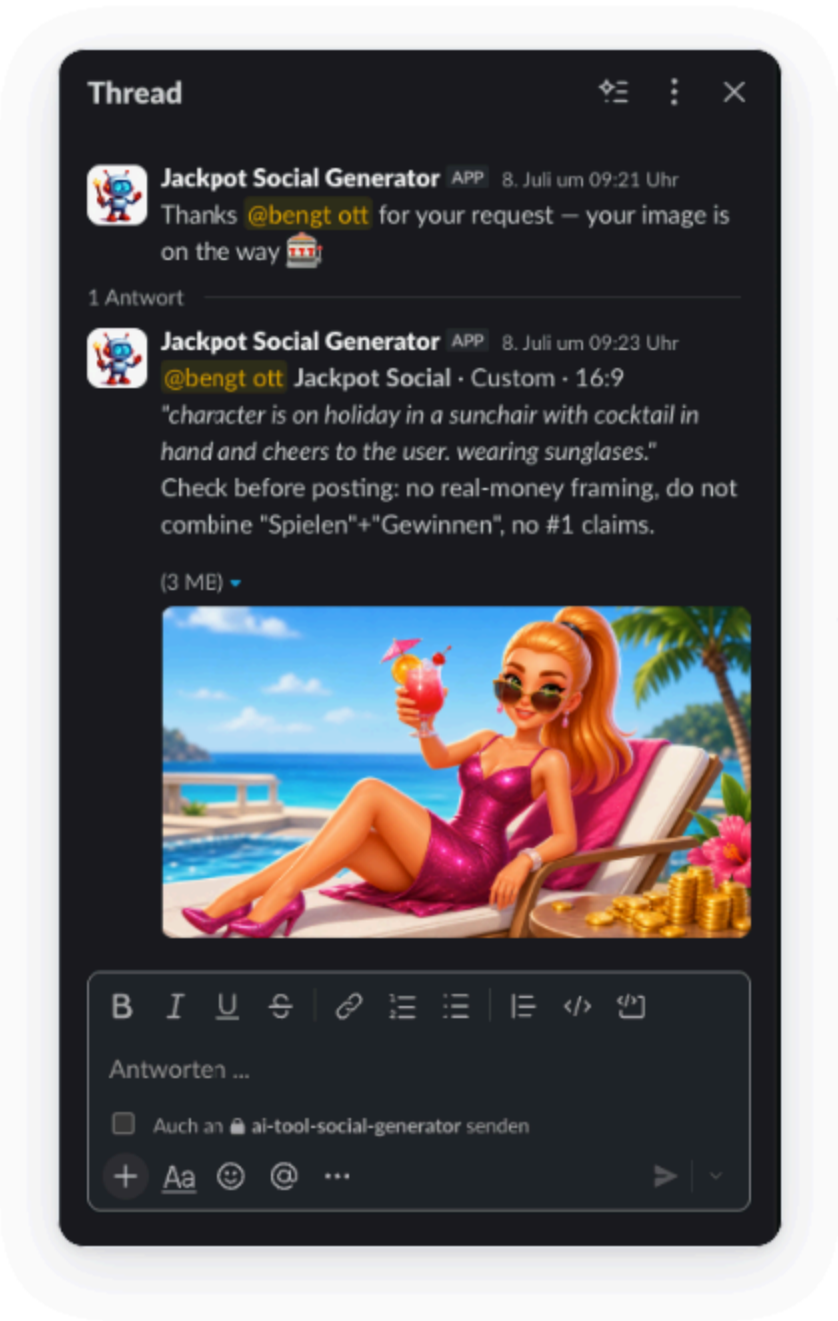
End-to-end architecture



Every result — the confirmation, each image, the final video — posts back into the same Slack thread as it finishes, tagging the requester.

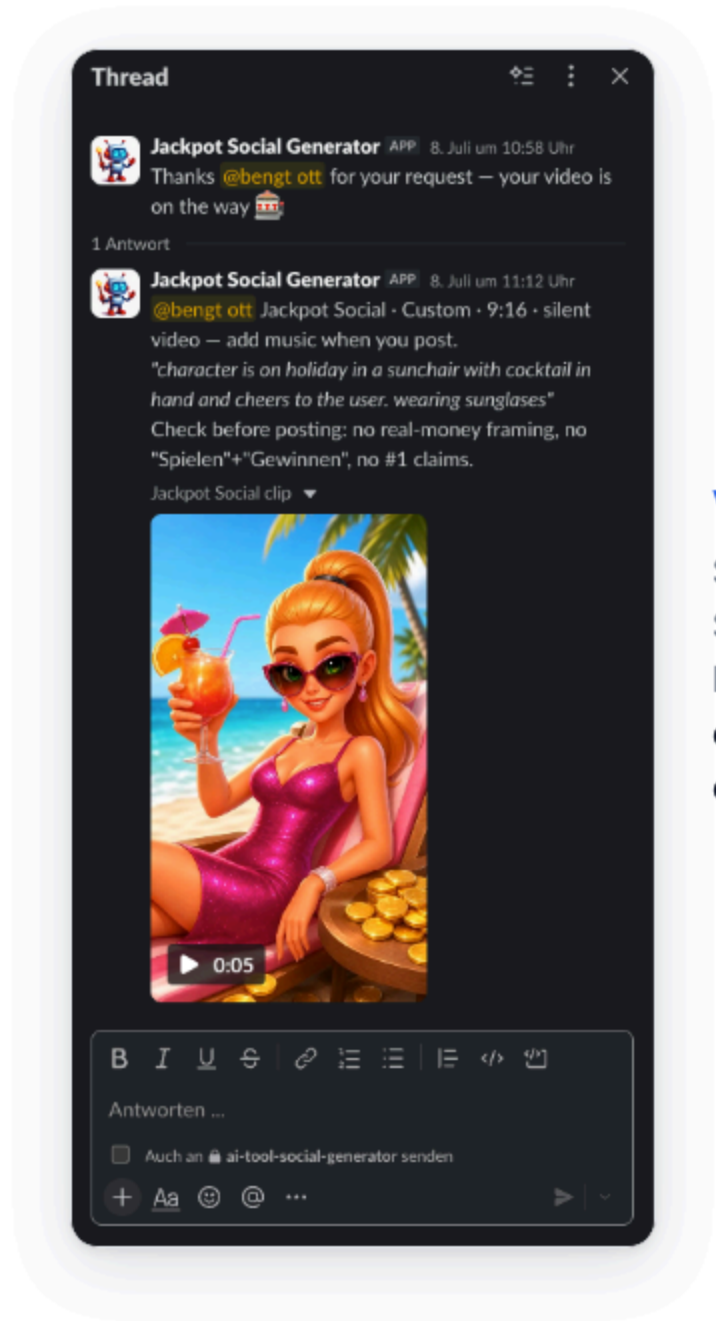
*One shape, two outputs: **Image** stops at the still after the QA gate. **Video** uses that approved still as frame 1 of the clip, which keeps the character consistent.*

What lands in the Slack thread



IMAGE

A confirmation, then the result as a threaded reply — friendly situation label, the requester's brief in italics, and a compliance cheat-sheet line.



VIDEO

Same flow, delivered as an inline Slack file — a 5 second silent clip. Frame 1 is the approved still, so the **character matches the image exactly**.

How a request runs, start to finish

Fully automatic — no approval checkpoints. Each step is a node that hands its output to the next; "done" means the node returned successfully.

- 1 **Modal submit** — structured input arrives (output type, situation, formats)
- 2 **Credit gate** — checks the account balance; blocks or warns before any spend
- 3 **Parse + compliance** — builds the prompt, applies social-casino rules, fans out one item per format
- 4 **Generate → QA → deliver** — per format: still image, quality check, then post to the thread
- ✓ No approval step — but every stage posts to Slack the moment it is ready



Typed input from the start

Because the request comes from a structured modal, every field — output type, situation, formats — is known before the run begins. The workflow never has to interpret free-text intent, which removes a whole class of routing errors.

04 · QUALITY GATE

A quality check before anything ships

Fully automatic delivery needs a floor. Before an image is posted, a vision model scores it against a 7-point checklist and can send it back to regenerate.

The 7 checks

character on-model · text clean · no artifacts · on-brand · aspect correct · faithful to brief · compliance-safe (calibrated for a social casino).

Retry then flag

On a fail, it regenerates with the failure reasons fed back into the prompt — up to 2 retries. Still failing after 3 attempts, the image is posted **flagged**, never silently.

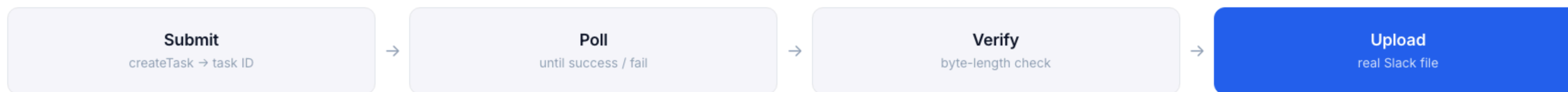


Fail-open by design

If the eval service itself is down, the gate delivers unflagged rather than blocking support. It raises the quality floor; it does not replace the human check before posting to Instagram or Facebook.

The step that takes minutes, made safe

Stills finish in seconds; video takes minutes. The pipeline treats "submit → poll → verify → upload" as one guarded unit of work per shot.



Hang guards

A stuck task is abandoned at ~90s and retried early, with a "still working" note at ~1 min — instead of waiting minutes for the API to eventually error.

Cross-model fallback

Image seed order is nano-banana → nano retry → GPT Image 2. An outage of either model switches silently instead of failing.

Bounded end-to-end

Worst case is capped at ~7–8 min per request; a workflow timeout is the final backstop. A global error handler alerts on any crash.

Where files live, and how they hand off

No database, no object storage. Each step passes a URL directly to the next — the generated still is fed straight into the video model as its input frame.



Reference

Two public character sheets (turnaround + expressions) read fresh on every run — the consistency anchor.



Still → video

The generated still is a hosted URL passed directly as the video seed. No re-upload, no queue between steps.



Final delivery

Images post as Slack image blocks; the finished video uploads as a **native Slack file**. Both render inline in the thread.



How results persist

The generation URLs are short-lived, but Slack fetches and re-hosts every image it renders through its own image proxy — so posted results keep showing in the thread long after the source URL expires (observed at 7+ days). Video is stored by Slack as a native file from the start.

Two people asking at once

Concurrent by default

Every request is an isolated n8n execution with its own state. Support members can submit in parallel and never collide — there is no shared in-memory object to contend over.

The shared resource

The one thing all requests share is the kie.ai credit pool. The credit gate protects it: a soft warning, then a hard stop before the balance runs dry.

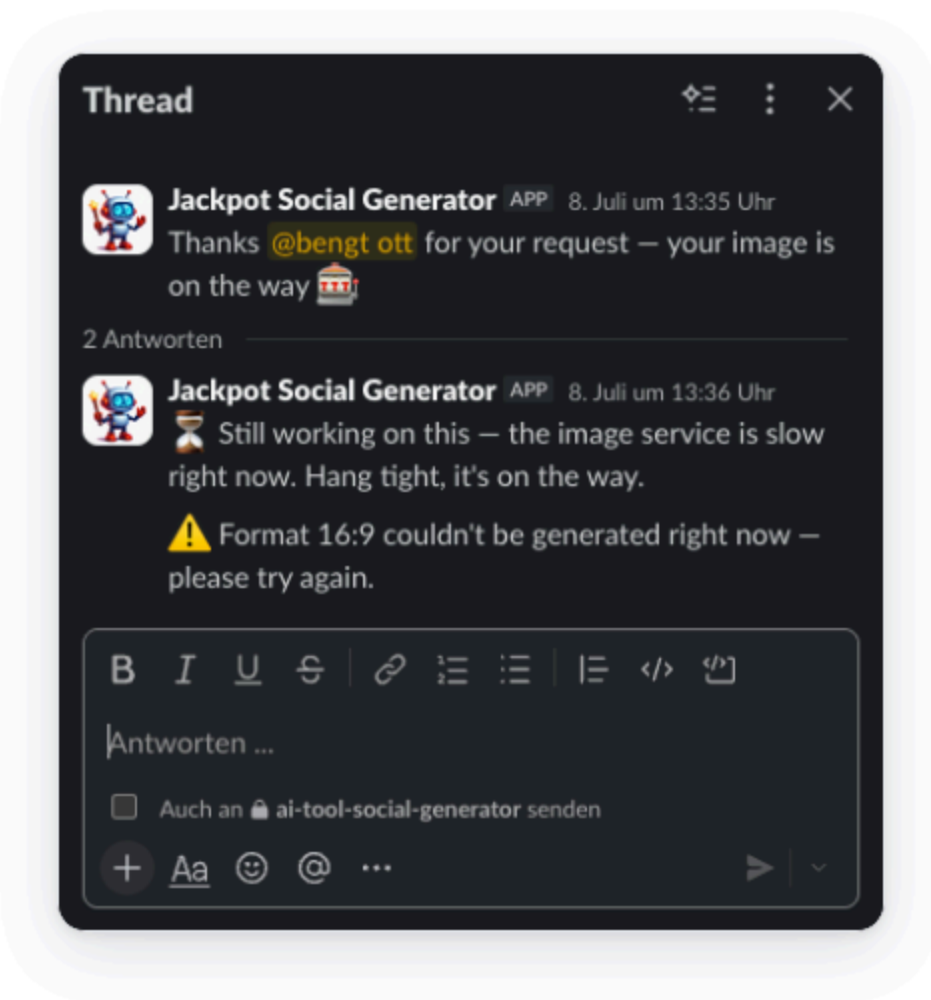


Access is gated too

A channel allowlist rejects any request from an unapproved channel before generation — so cost is bounded not just by budget, but by who can trigger a run at all.

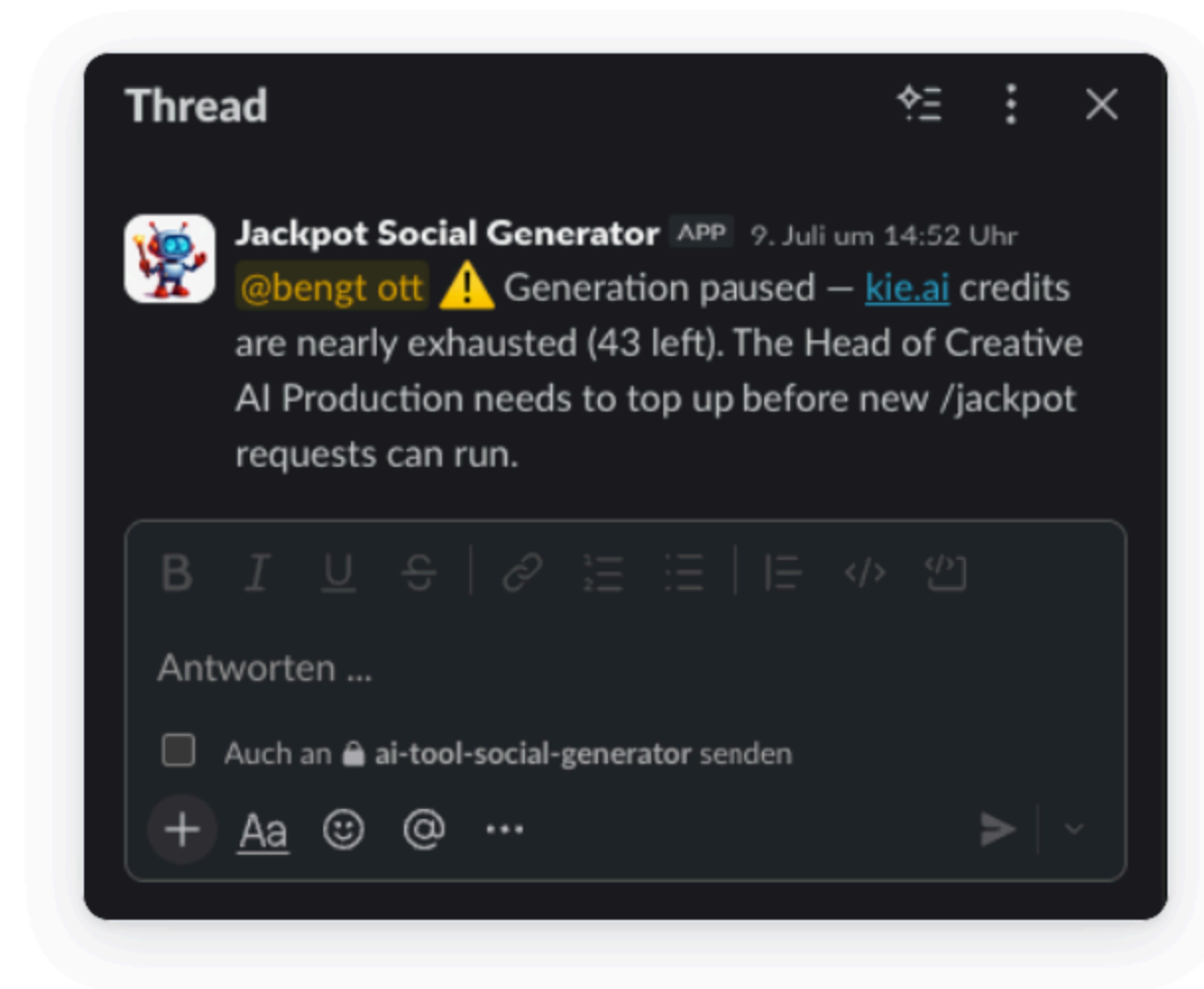
Status and guardrails, in the thread

The tool talks back in the same thread — so a slow render, a failed format, or an empty budget is never silent.



PROGRESS & GRACEFUL FAIL

A "still working" note when a render runs slow, and a clear per-format failure instead of a silent drop.



CREDIT STOP

The hard stop firing **before any spend** — generation is paused and the owner is told to top up.

SUMMARY

At a glance

Built with?

n8n Cloud + Slack + kie.ai — managed, no local codebase, no server to run.

Sequencing?

Structured modal → fully automatic fan-out. A self-QA gate instead of an approval step; every result posts to the thread live.

Files?

URLs passed directly step-to-step, no database. Video delivered as a permanent Slack file; image permanence on the roadmap.

Concurrency?

Isolated per execution by default — parallel requests, no shared state. Shared credit pool guarded by the credit gate.



Scope of this document: architecture and runtime behaviour. Prompt library, situation presets, and credential setup are covered separately in the tool's maintenance reference.

